

Sistema de Ficheros VFS de Linux

Diseño de Sistemas Operativos

Grado en Ing. de Computadores, 3er Curso

Depto. de Arquitectura de Computadores
Universidad de Málaga

Contenidos

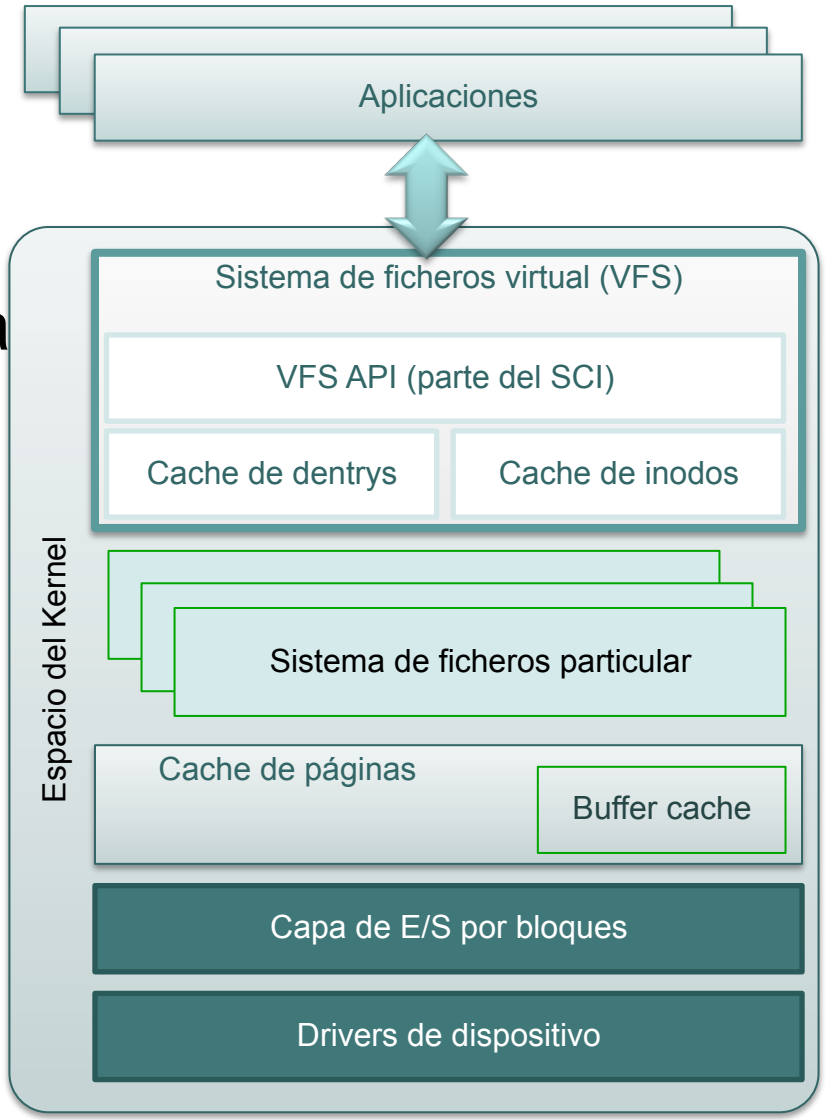
- Sistema de ficheros virtual
- Ficheros en disco
- Estructuras de datos del VFS y llamadas al sistema
  - Super bloque
  - I-nodos
  - Entradas de directorio
  - Cache de entradas de directorio
  - Ficheros
- Ficheros en memoria

Sistema de ficheros

- Es un almacenamiento jerárquico de datos
- Los datos están contenidos en ficheros (cadena ordenada de bytes)
- Los ficheros están organizados en directorios
- En linux un directorio se implementa como un fichero que contiene la lista de nombres (ficheros u otros directorios)
- En linux cada fichero tiene asociado en el disco una estructura inodo con:
  - Propiedades o metadatos (excepto el nombre que está en el directorio)
  - Punteros a los bloques de datos
- En linux además el sistema de ficheros tiene un super-bloque que almacena sus parámetros fundamentales en el disco.

Virtual Filesystem (VFS)

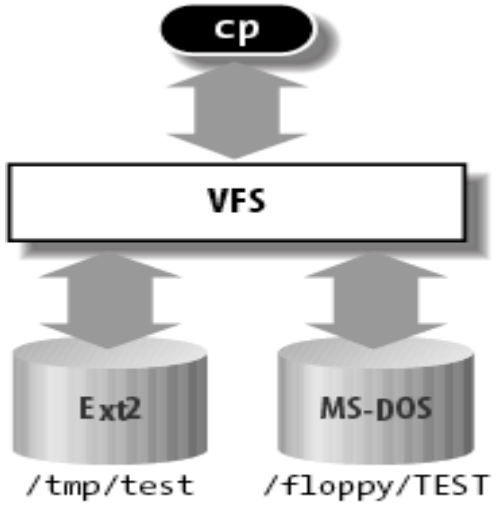
- Maneja las llamadas al sistema para acceso al sistema de ficheros
- Presenta un interfaz común para acceder a diferentes FS
- Permite interactuar con diferentes sistemas de ficheros
  - Disco (Ext3, NTFS, HFS,…)
  - Red (NFS, CIFS,…)
  - Virtuales (/proc)



Ejemplo de uso

- Copiamos desde un disco FAT a uno Ext2

```
$ cp /floppy/TEST /tmp/test
```



(a)

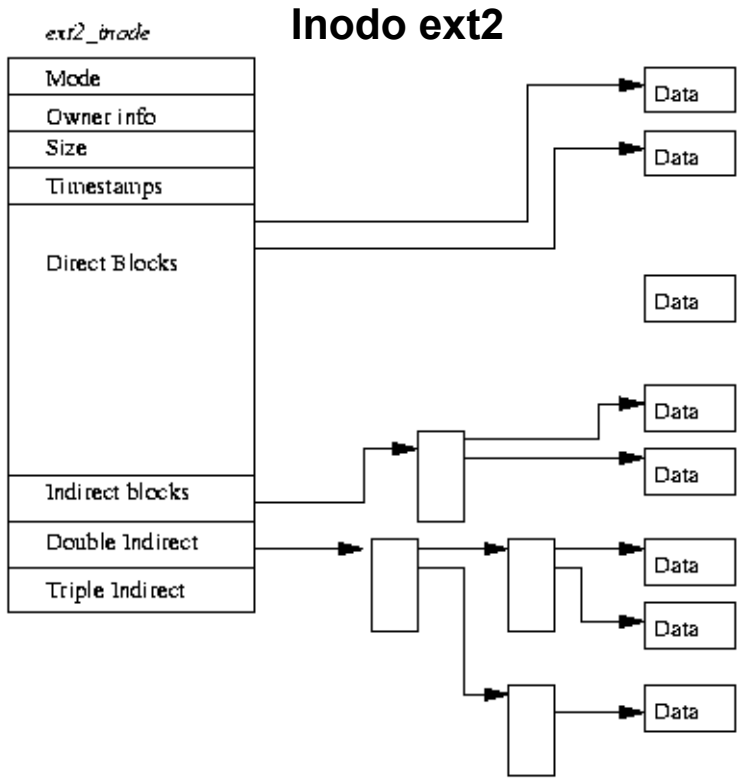
```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
       O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

(b)

Ficheros en disco



- El VFS usa los objetos básicos que encontramos en un sistema de ficheros Linux
- Otros sistemas de ficheros tendrán que mapear sus estructuras físicas al modelo de Linux (VFS)
- El sistema de ficheros nativo de Linux tendrá un mejor rendimiento o al menos será más fácil de manejar

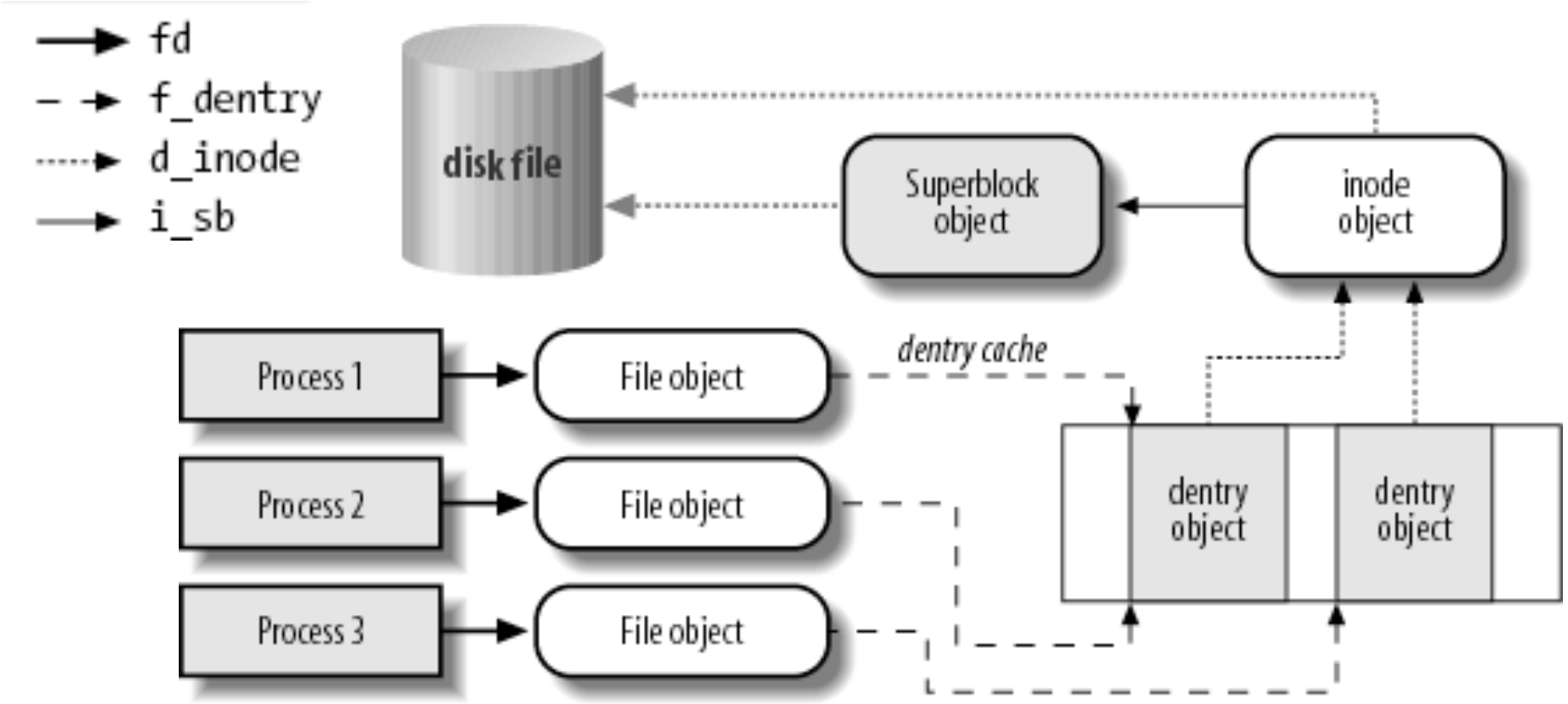


Estructuras de datos del VFS

- Para representar el sistema de ficheros VFS usa 4 objetos/estructuras en el kernel:
  - Superbloque (superblock)
    - Representa un sistema de ficheros montado
  - Inodo (inode - index node)
    - Representa a un fichero en disco
  - Entrada de directorio (dentry)
    - Representa un elemento de un camino (path)
  - Fichero (File)
    - Representa un fichero abierto por un proceso

Estructuras de datos del VFS

Relación entre las diferentes estructuras:



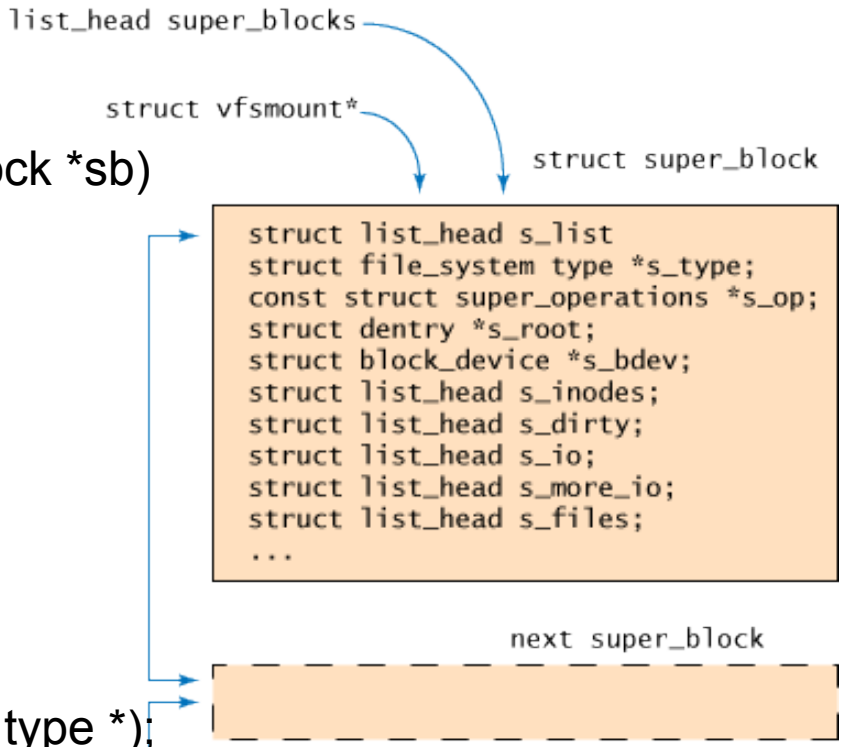
VFS orientado a objetos

- Cada objeto del VFS tiene un conjunto de métodos asociados
  - Los ficheros tienen una tabla f_op con punteros a todas las operaciones de fichero: read, write, open, flush,…
- Cada vez que se instancia un nuevo objeto, el VFS le enlaza las funciones apropiadas para su sistema de ficheros en concreto
  - Cuando se crea un nuevo objeto fichero (se abre un fichero nuevo) se le colocan las llamadas proporcionadas por su sistema de ficheros
- Un ejemplo:
  - Una llamada al sistema read(…) para el fichero file1 se transforma en file1->f_op->read(…), que contiene la implementación válida de read para el sistema de ficheros al que pertenece el fichero file1

Super bloque

- Describe el sistema de ficheros (montado)
- Corresponde con el super bloque del disco

struct **super_operations**
{
struct inode * **alloc_inode**(struct super_block *sb)
void **dirty_inode**(…)
int **sync_fs**(…)
int **statfs**(…)
int **remount_fs**(…)
…
}



Registro y montaje de FS
/proc/filesystems
int **register_filesystem**(struct file_system_type *);
int **unregister_filesystem**(struct file_system_type *);
Montaje y desmontaje: **get_sb**() / **kill_sb**()

Inodos

- Representa un fichero en disco

struct inode_operations {
  int (***create**) (struct inode *,struct dentry
  struct dentry * (***lookup**) (struct inode *,s
  int (***link**) (struct dentry *,struct inode *,s
  int (***unlink**) (struct inode *,struct dentry
  int (***mkdir**) (struct inode *,struct dentry *,int);
  int (***rmdir**) (struct inode *,struct dentry *);
  int (***mknod**) (struct inode *,struct dentry *,int,dev_t);
  int (***rename**) (struct inode *, struct dentry *, struct inode *, struct dentry *);
  int (***permission**) (struct inode *, int);
  int (***setattr**) (struct dentry *, struct iattr *);
  int (***getattr**) (struct vfsmount *mnt, struct dentry *, struct kstat *);
  …};

struct inode

```
struct list_head i_dentry;
struct timespec i_atime;
struct timespec i_mtime;
struct timespec i_ctime;
gid_t i_gid;
uid_t i_uid;
loff_t i_size;
const struct file_operations *i_fop;
const struct inode_operations *i_op;
struct address_space *i_mapping;
struct address_space *i_data;
...
```

Dentry (entrada de directorio)

```
struct dentry_operations {
        int (*d_revalidate) (struct dentry *, struct
        int (*d_hash) (struct dentry *, struct qstr
        int (*d_compare) (struct dentry *, struct
        int (*d_delete) (struct dentry *);
        void (*d_release) (struct dentry *);
        void (*d_iput) (struct dentry *, struct inode *);
        char *(*d_dname) (struct dentry *, char *, int);
… };
```

struct dentry

```
struct super_block *d_sb;
struct dentry *d_parent;
struct list_head d_subdirs;
struct dentry_operations *d_op;
unsigned char d_iname[];
struct inod *d_inode;
...
```

- **Estado** de la entrada de directorio (USED, UNUSED, NEGATIVE)
- **Cache** de Dentrys - dcache (y cache de inodos - icache)
  - Tabla hash para localizar los paths rápidamente
  - Lista de dentrys no usadas o negativas (LRU) listas para ser liberadas
  - Las dentrys no usadas se conservan en la cache (contador de uso =0)
  - Las negativas también se conservan!!
  - Los inodos de dentrys cacheadas también se cachean

Cache de Dentry (dcache)

- Un ejemplo: trabajamos con el fichero

  /home/pepe/proyecto/programa.c

- Cada vez que se accede al fichero (abrirlo, modificarlo, compilar,…) hay que resolver y localizar diferentes directorios en el camino “/”, “home”, “pepe”, “proyecto”, y finalmente el fichero “programa.c”
- Para evitar tener que recorrer estos caminos repetidamente, el kernel almacena los dentry (paths) que visita en una cache
- Antes de buscar el camino en disco se consulta la cache (tabla hash) y así comprobamos si ya hemos visitado ese path antes y ya lo tenemos resuelto
- Se mantiene una copia en memoria de los inodos con dentry cacheados (esta es la caché de inodos, icache)

Ficheros (File object)

- Cada vez que se abre un fichero (open) se crea este objeto
  - Se puede abrir un fichero dos veces
- Apunta un dentry que a su vez apunta a un inodo (estos objetos son únicos)
- No representa un fichero en disco (eso es el inodo)
- El objeto fichero desaparece con close()
  - ¿o no? (ficheros compartidos por procesos a través de fork)

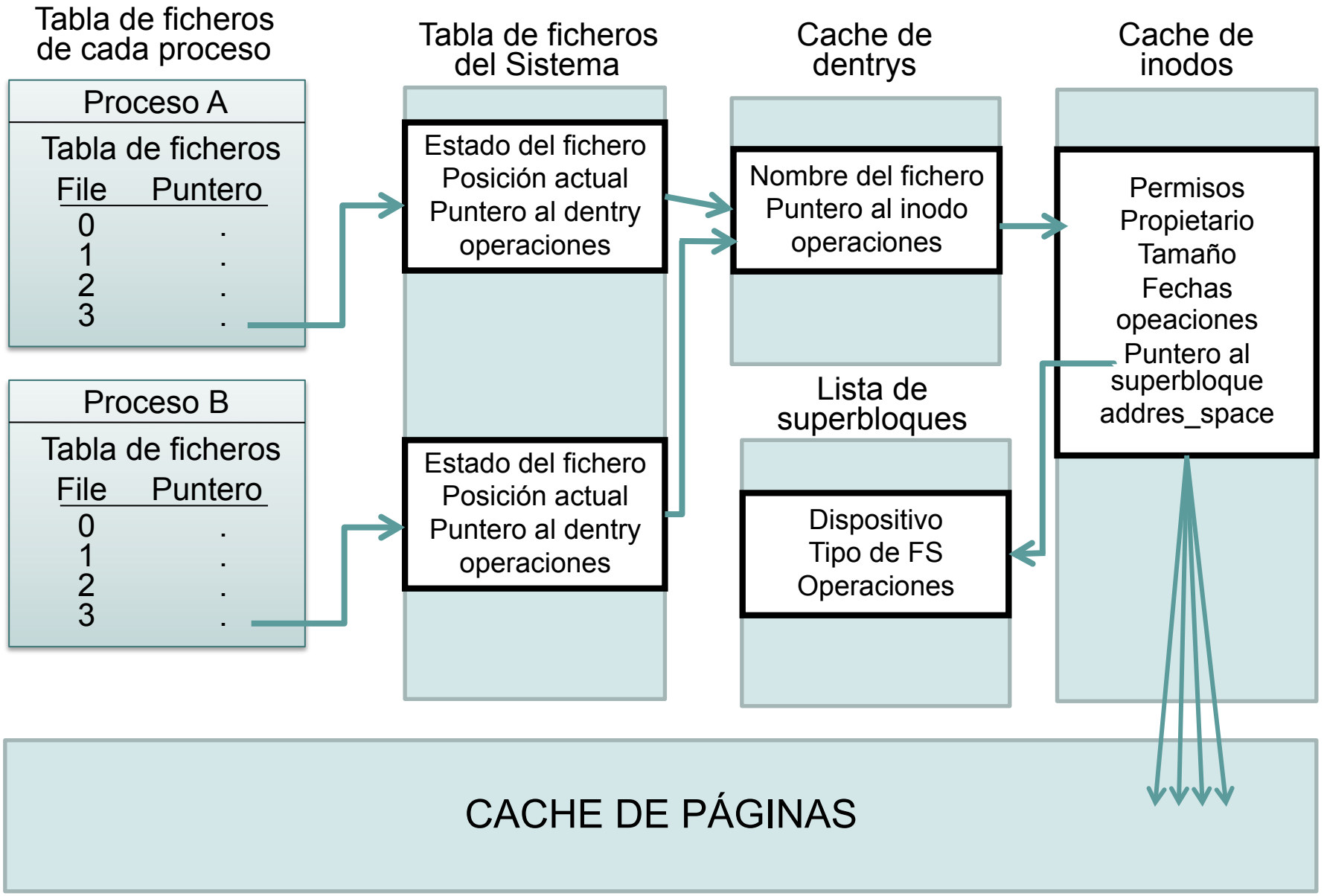
Ficheros

struct file

```
struct path f_path;
struct dentry (f_path.dentry);
const struct file_operations *f_op;
unsigned int f_flags;
fmode_t f_mode;
lodd_t f_pos;
...
```

```
struct file_operations {
        struct module *owner
        loff_t (*llseek) (struct file *, loff_t, int);
        ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
        ssize_t (*write) (struct file *, const char __user *, size…);
        int (*readdir) (struct file *, void *, filldir_t);
        int (*open) (struct inode *, struct file *);
        int (*flush) (struct file *, fl_owner_t id);
        int (*release) (struct inode *, struct file *);
        int (*fsync) (struct file *, struct dentry *, int datasync);
… };
```

Ficheros en memoria



Lecturas

- Understanding the Linux Kernel:
  - Capítulo 12. The Virtual Filesystem
- Linux Kernel Development:
  - Capítulo 15. The Virtual Filesystem
- Artículos enlazados en el C.V.
  - Anatomy of the Linux virtual file system switch

Ejercicio: Montando un sistema de ficheros

$ **dd if=/dev/zero of=file.img bs=1k count=10000**
10000+0 records in
10000+0 records out $
$ sudo **losetup /dev/loop0 file.img**
$ sudo **mke2fs -c /dev/loop0 10000**
mke2fs 1.41.11 (14-Mar-2010)
…
$ sudo **mkdir /mnt/point1**
$ sudo **mount -t ext2 /dev/loop0 /mnt/point1**
$ **ls /mnt/point1**
lost+found
$

Ver artículo: “Anatomy of the Linux file system”